AI Usage Journal

BudgetLang DSL Project — Part 1

03 May 2026 — 08 May 2026

20 Entries · 4 Days · Part 1 Submission

Table of Contents

03 May 2026

Note: This entry was made before the general form was established.

Entry 1

Date	2026-05-03
Phase	**Writing**
AI tool	Claude (unspecified)
Goal	Decide whether to use chapter MD files to assist the AI during the project.

PROMPT

is it a good idea to have chapter from 1 to 7 as md file to make AI help me while i am doing the project to follow the book?

RESPONSE (key part)

The AI confirmed this was a great idea with a specific approach:

- The AI cannot read the physical textbook, so having chapters as .md files lets it reference exact Sebesta terminology and section numbers for D1 and exam prep.
- Recommended creating concise summaries (not full copies) with: key definitions, section numbers, design questions, and important examples.
- Suggested folder structure: `Sebesta_References/` with files like `Ch1_Preliminaries.md`, `Ch5_Names_Bindings_Scope.md`, etc.
- Bonus: writing the summaries yourself doubles as exam preparation for the 3-sheet allowance.

ACCEPTED

The idea was accepted — concise chapter reference files were planned to help the AI and serve as exam cheat-sheet material.

REJECTED / MODIFIED

None.

ERRORS YOU CAUGHT

None.

REFLECTION

Giving the AI access to textbook summaries makes its suggestions more accurate and aligned with the course terminology.

Entry 2

Date	2026-05-03
Phase	**Writing**
AI tool	Claude (unspecified)

Goal	Write a reusable prompt to send to another AI to convert Sebesta chapter PDFs into structured MD reference files.

PROMPT

i will give all the pdf to one AI to do them as md files i want u to write me a prompts that i got them in proper way

RESPONSE (key part)

The AI produced a detailed prompt template specifying the exact structure for each chapter MD file:

- Sections: Key Concepts (with §section numbers), Definitions table, Design Questions, Important Examples, Quick Reference Table.
- Rules: concise and scannable, always include §X.Y, use Sebesta's exact terms, 2-4 pages per file.
- Named output files: `Ch1_Preliminaries.md`, `Ch3_Syntax.md`, `Ch4_Lexical_Semantic.md`, `Ch5_Names_Bindings_Scope.md`, `Ch6_Data_Types.md`, `Ch7_Expressions.md`.

ACCEPTED

The prompt template was accepted and used to generate the chapter reference files.

REJECTED / MODIFIED

None.

ERRORS YOU CAUGHT

None.

REFLECTION

Writing a precise meta-prompt saves time when delegating repetitive structured tasks to AI.

---

05 May 2026

Entry 1 of 6

Date	2026-05-05
Phase	**Writing**
AI tool	Claude Opus 4.6 (Thinking)
Goal	Update the agent config files (AGENTS.md, CLAUDE.md, AGENT.MD) to match the AI Usage Journal format required by the project handout, and rename the logging folder from LLM_Chat to AI_Usage_Journal.

PROMPT

in the Project_Handout.md file there is something called The AI Usage Journal i think it is the same of the LLM_Chat idea i have but i think i miss some data i should add them can i fix the AGENTS.MD, AGENTS.md, CLAUDE.md

(followed by: okay i want to rename the folder LLM_Chat as the name required)

RESPONSE (key part)

The AI compared the existing LLM_Chat logging format against the project handout's §6 and §8 requirements and identified 7 missing fields: **Phase**, **AI tool**, **Goal**, **Accepted**, **Rejected/Modified**, **Errors You Caught**, and **Reflection**. It also noted the missing minimum entry counts (4 for Part 1, 6 for Part 2) and mandatory experiments (E1, E2, E3).

The AI then:

1. Rewrote all three agent files (AGENTS.md, CLAUDE.md, AGENT.MD) with the full entry template matching the handout's §8 table.
2. Renamed `src/LLM_Chat/` → `src/AI_Usage_Journal/` to match the D4 deliverable name.
3. Updated all folder path references in the agent files accordingly.

ACCEPTED

All changes were accepted — the new template now includes every field from the handout's entry template (§8), the folder name matches the deliverable name, and the grading criteria reminders were added.

REJECTED / MODIFIED

None rejected. The AI's updates were a direct mapping from the handout requirements.

ERRORS YOU CAUGHT

The AI's first attempt to update AGENTS.md partially failed (one replacement chunk couldn't find its target string because the file had been modified by user_rules injection). This was caught and fixed with a follow-up edit.

REFLECTION

This was a straightforward alignment task — making sure the logging format matches the professor's rubric exactly. The key takeaway is that small formatting gaps (missing fields like "Accepted" and "Reflection") could cost points on D4 since those are the graded parts of the journal. Always cross-reference deliverable specs early.

Entry 2 of 6

Date	2026-05-05
Phase	**Writing**
AI tool	Claude Opus 4.6 (Thinking)
Goal	Update the agent config files so that the AI asks the user to write the "Errors You Caught" and "Reflection" sections themselves, instead of the AI filling them in automatically.

PROMPT

Errors you caught | Reflection (2–3 sentences)

in these part of the AL_Usage_Journal files as me after every time i says to u add this prompts to the AL_Usage_Journal ask me to give u a text for Errors you caught and after that take the text if it has any grammar mistake fix them and add them same thing for the Reflection (2–3 sentences)

every time show me what each one mean:

Errors you caught: Any mistakes the AI made. How did you detect them? What would have happened if you hadn't?

Reflection (2–3 sentences): What did you learn about the problem — or about how to use AI — from this session?

RESPONSE (key part)

The AI added **rule 8** to all three agent files (AGENTS.md, CLAUDE.md, AGENT.MD) titled "User-written Errors & Reflection". The rule specifies that when logging, the AI must:

1. Show the user the definitions of both fields.
2. Ask the user to provide their own text for "Errors You Caught".
3. Ask the user to provide their own text for "Reflection".
4. Fix any grammar mistakes before logging.

ACCEPTED

The new workflow rule was accepted — it ensures the professor sees student-written reflections, not AI-generated ones.

REJECTED / MODIFIED

None rejected.

ERRORS YOU CAUGHT

The mistake I found was that the AI was not asking me to fill in the "Errors You Caught" section — it was filling it by itself from the chat. However, the professor requires that the student writes this section, not the AI.

REFLECTION

I am learning how to create agents for AI to make it more helpful in my workflow.

Entry 3 of 6

Date	2026-05-05
Phase	**Design**
AI tool	Claude Opus 4.6 (Thinking)
Goal	Understand what needs to be done for Part 1 of the project and confirm the first steps.

PROMPT

okay i want to start working on part 1 for the project: u can see Project_Handout.md & Submission Guide.md after i prepare some things will help me i want to know what should i do now. i think i have to choice the DSL and the programming language i want to program with now right?

RESPONSE (key part)

The AI confirmed that the first step is to choose the DSL domain and the implementation language. It summarized the Part 1 deliverables (D1–D5) due 8 May and listed the minimum DSL requirements from the handout: at least 3 primitive types + 1 structured type, at least 1 control structure beyond sequencing, user-defined functions/procedures, at least 1 domain-specific construct, and a nontrivial expression language.

ACCEPTED

The summary and action plan were accepted — it confirmed the correct first steps.

REJECTED / MODIFIED

None rejected.

ERRORS YOU CAUGHT

None.

REFLECTION

I am still choosing the DSL and the programming language I will use for it.

Entry 4 of 6

Date	2026-05-05
Phase	**Design**
AI tool	Claude Opus 4.6 (Thinking)
Goal	Brainstorm unique DSL domain ideas for the project.

PROMPT

can u give me some ideas

RESPONSE (key part)

The AI suggested 7 unique DSL domains not listed in the handout: **ArcadeLang** (2D game levels), **BudgetLang** (personal finance), **FitLang** (workout routines), **RouteLang** (navigation), **SurveyLang** (quiz builder), **RoboLang** (robot controller), and **GardenLang** (garden planner). Each suggestion included the 3+ primitive types, a structured type, and a domain-specific construct. The AI also recommended Python or JavaScript as the implementation language for the tight timeline.

ACCEPTED

The list of ideas was helpful for narrowing down options. BudgetLang stood out as a natural fit.

REJECTED / MODIFIED

None rejected.

ERRORS YOU CAUGHT

None.

REFLECTION

I was already thinking about budget and similar topics; the other ideas look a bit complex.

Entry 5 of 6

Date	2026-05-05
Phase	**Design**
AI tool	Claude Opus 4.6 (Thinking)
Goal	Get a detailed overview of BudgetLang features and confirm C# as the implementation language.

PROMPT

i think i will go with budgetLang but i want to look about more details but for the programming language no python is not a good thing for me i think i will go with C#

RESPONSE (key part)

The AI provided a detailed BudgetLang design including: 3 primitive types (`money`, `percent`, `string`), a `transaction` structured type (record with amount, category, type fields), control structures (`if/else`, `foreach`,

`when`), and domain-specific constructs (`budget`, `track`, `report`, `alert`). It also listed interesting design questions for the exam: mixed-mode arithmetic (`money * percent`), type equivalence, short-circuit evaluation, and whether assignment is a statement or expression. A sample program was included.

ACCEPTED

The detailed overview was accepted. C# was confirmed as the implementation language since Python is not familiar to me.

REJECTED / MODIFIED

Rejected Python — I have no experience with it. Chose C# instead, which I know from other projects.

ERRORS YOU CAUGHT

None — we are discussing the DSL and the language I am using to program.

REFLECTION

AI helps so well, but I have to check every suggestion it gives me. For example, it suggested Python, but I am not good at Python and I have never used it, so I chose something better for me.

Entry 6 of 6

Date	2026-05-05
Phase	**Design**
AI tool	Claude Opus 4.6 (Thinking)
Goal	Clarify whether target, invest, income, and expense should be types or something else in BudgetLang.

PROMPT

i think we can add some other type too: example target to buy | invest. is these a good idea and are they consider as a type? and income, expense will be a type too?

RESPONSE (key part)

The AI explained that `income` and `expense` are **not** separate types — they are values of an **enumeration type** (like a C# enum: `transaction_kind = { income, expense }`). It suggested `target` (savings goal) as a second **structured type** (record with name, goal, saved, deadline fields), and `invest` as a **domain-specific command** rather than a type. The final suggested type system: 5 primitives (`money`, `percent`, `int`, `string`, `bool`), 1 enumeration (`transaction_kind`), and 2 structured types (`transaction`, `target`).

ACCEPTED

The type system suggestion was accepted as a starting point, with the understanding that it can be changed later.

REJECTED / MODIFIED

None rejected for now — the design is still flexible.

ERRORS YOU CAUGHT

None up to now.

REFLECTION

The AI gave me some ideas and showed me how they would work, what I could do, and how it would look. This made me understand the project more.

---

07 May 2026

Entry 1 of 8

Date	2026-05-07
Phase	**Design**
AI tool	Codex (GPT-5)
Goal	Validate the minimum BudgetLang core features for Part 1 before moving on to the sample program, tokens, and grammar.

PROMPT

okay lest start working on part 1 what do i need to do now

no

lets Finalize the language core in one page.

Decide the minimum features:

Primitive types: money, percent, int, string, bool

Structured type: transaction

Control structures: if, foreach, switch

Functions

Domain-specific commands: budget, track

Expressions: arithmetic, comparisons, boolean ops

are these good up to now ?

RESPONSE (key part)

The AI said the proposed scope was a solid, small Part 1 core and recommended locking it as:

- Primitive types: `money`, `percent`, `int`, `string`, `bool`
- Structured type: `transaction`
- Control structures: `if`, `foreach`, `switch`
- Functions with typed parameters and a single return value
- Domain-specific commands: `budget`, `track`
- Expressions: arithmetic, comparison, and boolean expressions

It also recommended keeping `transaction`, `switch`, and functions simple so the parser would stay manageable for the deadline.

ACCEPTED

I accepted the suggested core as the working BudgetLang scope for Part 1 because it is small enough to implement but still satisfies the handout requirements for types, a structured type, control structures, functions, domain-specific constructs, and expressions.

REJECTED / MODIFIED

I kept the feature set minimal and did not add extra domain features yet. I also treated the advice about dropping `switch` as optional and kept it for now, but only in a simple form.

ERRORS YOU CAUGHT

Up to now, I see the AI suggestions are good.

REFLECTION

I am starting to define the basics. These include types, statements, and expressions.

Entry 2 of 8

Date	2026-05-07
Phase	**Writing**
AI tool	Claude Opus 4.6 (Thinking)
Goal	Add a rule to all agent config files requiring the AI to follow Sebesta's textbook for all suggestions, using the book chapters in the project.

PROMPT

i want u to add to Agents.md , Agents.MD, claude.md that any thing as suggestion or i give it to him have to follow the book which the filles are in the book_chapter folder there is a MD-File-version if i did not find any thing about it in that fokder it can look later to the PDF-File-Version folder he can find the real book there

RESPONSE (key part)

The AI added **rule 9** ("Follow the textbook (Sebesta)") to all three agent files (AGENTS.md, AGENT.MD, CLAUDE.md). The rule states that all suggestions, design decisions, terminology, and explanations must be grounded in Sebesta's "Concepts of Programming Languages" (12th ed.). The lookup priority is: first check `src/book_chapters/MD-File-Version/` for markdown summaries, and if the needed information is not found there, then check `src/book_chapters/PDF-File-Version/` for the original full textbook chapters. The rule also requires citing section numbers (e.g., §5.5, §6.14) and prohibits inventing terminology that contradicts the book.

ACCEPTED

The new rule was accepted — it ensures all future AI responses are grounded in the course textbook rather than general knowledge.

REJECTED / MODIFIED

None rejected.

ERRORS YOU CAUGHT

The AI, when giving suggestions, did not follow our textbook (Sebesta). It would provide terminology or design rationale without grounding it in the book's chapters.

REFLECTION

I learned that I can make the AI follow the rules and structure I want by giving it specific sources to reference. Every time, it can check those sources and give suggestions or answers based on them.

Entry 3 of 8

Date	2026-05-07
Phase	**Design**

AI tool	Codex (GPT-5)
Goal	Confirm the starter BudgetLang core so it can be used as the fixed scope for the sample program, tokens, and grammar.

PROMPT

okay then lets say this will be our starter BudgetLang Core

Types:

money, percent, int, string, bool

transaction { amount: money, category: string, kind: string, note: string }

Statements:

variable declaration

assignment

if / else

foreach

switch

function declaration

return

budget

track

expression statement / function call

Expressions:

+ , -, *, /

```
<, <=, >, >=, ==, !=
&&, ||, !
```

parentheses

identifiers

literals

function calls

field access on transaction

RESPONSE (key part)

The AI confirmed that this was a good starter BudgetLang core for Part 1 because it was small enough to implement and still strong enough to satisfy the handout. It also said the next design step should be to define the concrete syntax for variable declarations, transaction values, `budget` and `track` statements, `foreach`, `switch`, and literal forms for `money` and `percent`, then turn that into a sample program, token list, and EBNF grammar.

ACCEPTED

I accepted this BudgetLang core as the working scope because it gives me a clear starting point for the grammar and parser without making the language too large for Part 1.

REJECTED / MODIFIED

No features were added beyond this starter core. I kept the scope fixed so I would not expand the language before defining the syntax.

ERRORS YOU CAUGHT

Nothing up to now.

REFLECTION

I am learning how the workflow is going, and I am learning how to do it. This helped me see how to move from ideas into a concrete language design.

Entry 4 of 8

Date	2026-05-07
Phase	**Design**
AI tool	Codex (GPT-5)
Goal	Improve the draft syntax for creating and tracking a transaction so it is clearer and easier to parse.

PROMPT

what do u mean abou this :

```
if (spent > rentLimit) {
    track transaction {
        amount: $50,
        category: "warning",
        kind: "expense",
        note: "Over budget"
    };
} else {
    budget "Status" spent;
}
```

is it a good idea to make it like this

track mylife

```
mylife.amount = 50;
string mylife.category = "warning"
string mylife.kind = "expense"
string mylife.note = "Over budget"

if (spent > rentLimit) {
    transaction mylife;

mylife.amount = $50;
mylife.category = "warning";
mylife.kind = "expense";
mylife.note = "Over budget";

track mylife;

} else {
    budget "Status" spent;
}
```

do u mean like this ?

RESPONSE (key part)

The AI first explained that `track transaction { ... }` was only a draft idea showing how `if / else`, `track`, and `transaction` could work together. It then said a cleaner and more parser-friendly version would be to declare a `transaction` variable first, assign its fields one by one, and call `track` after the record was fully built:

```
transaction mylife;
mylife.amount = $50;
mylife.category = "warning";
mylife.kind = "expense";
mylife.note = "Over budget";
track mylife;
```

The AI confirmed that this revised version was clearer, more readable, and easier to parse than the inline `track transaction { ... }` form.

ACCEPTED

I accepted the revised syntax because declaring a transaction first and then assigning its fields is easier to read and easier to explain in the grammar and parser design.

REJECTED / MODIFIED

I modified the AI's original small draft sample into a better version that looks closer to familiar programming languages. I also avoided redeclaring the field types during field assignment.

ERRORS YOU CAUGHT

The AI suggested a small draft sample, and I modified it to make it better. It is more readable like this and looks closer to familiar languages.

REFLECTION

AI can give a good suggestion, but I can still make it better. This showed me that I should treat the AI's draft as a starting point and improve the syntax based on readability and clarity.

Entry 5 of 8

Date	2026-05-07
Phase	**Design**
AI tool	Codex (GPT-5)
Goal	Document how the original BudgetLang sample program was improved into a more readable and more familiar syntax.

PROMPT

okay i want u to show in AI_Usage_Journal that u suggest this to me

```
function money totalExpenses(transaction[] items) {
    money total = $0;

    foreach (transaction t in items) {
        if (t.kind == "expense") {
            total = total + t.amount;
        }
    }

    return total;
}
```

```
money monthlyIncome = $3000;
money rentLimit = $1200;

transaction rent = transaction {
    amount: $1100,
    category: "housing",
    kind: "expense",
    note: "May rent"
};

transaction salary = transaction {
    amount: $3000,
    category: "job",
    kind: "income",
    note: "Monthly salary"
};

budget "Rent" rentLimit;
track rent;
track salary;

transaction[] records = [rent, salary];

money spent = totalExpenses(records);

if (spent > rentLimit) {
    track transaction {
        amount: $50,
        category: "warning",
        kind: "expense",
        note: "Over budget"
    };
} else {
    budget "Status" spent;
}
```

and how i modify it to this and

```
money totalExpenses(transaction[] items) {
    money total = $0;

    foreach (transaction t in items) {
        if (t.kind == "expense") {
            total = total + t.amount;
        }
    }

    return total;
}

money monthlyIncome = $3000;
money rentLimit = $1200;

transaction rent;

rent.amount = $1100;
rent.category = "housing";
rent.kind = "expense";
rent.note = "May rent";
```

```
transaction salary;

salary.amount = $3000;
salary.category = "job";
salary.kind = "income";
salary.note = "Monthly salary";


budget "Rent" rentLimit;
track rent;
track salary;

transaction[] records = [rent, salary];

money spent = totalExpenses(records);

if (spent > rentLimit) {
    transaction mylife;

    mylife.amount = $50;
    mylife.category = "warning";
    mylife.kind = "expense";
    mylife.note = "Over budget";

    track mylife;
} else {
    budget "Status" spent;
}
```

RESPONSE (key part)

The AI said this could be logged as a before-and-after design change. It identified the original AI-suggested version as using two patterns:

- `function money totalExpenses(...)`
- inline transaction construction such as `transaction rent = transaction { ... }` and `track transaction { ... }`

It then described the improved version made by the user:

- `money totalExpenses(...)` without the `function` keyword
- `transaction rent;` and `transaction salary;` followed by field-by-field assignment
- `transaction mylife; ... track mylife;` instead of inline `track transaction { ... }`

The AI said this change made the sample program more readable, more familiar, and easier to parse.

ACCEPTED

I accepted the improved version of the sample program because it is clearer and looks closer to familiar programming languages. It also seems easier to describe in the grammar and easier to implement in the parser.

REJECTED / MODIFIED

I modified the AI's original syntax in two main ways. I removed the `function` keyword from the function declaration style, and I replaced inline transaction construction with separate declaration and field assignment.

ERRORS YOU CAUGHT

The AI suggestion was good, but it was not good enough. I modified it to be more readable and better.

REFLECTION

AI can also check whether my idea is a good change or a bad one. This helped me use the AI as a reviewer for my design decisions instead of only taking its first suggestion.

Entry 6 of 8

Date	2026-05-07
Phase	**Lexer**
AI tool	Codex (GPT-5)
Goal	Review the lexical/token classification file and identify any missing or incorrect token categories.

PROMPT

okay i did it all in one text file [lexical&Token.txt](src/lexical&Token.txt)

there is some stuff i did not include the token i am not sure for example :

"....." i think it is a literal

can u check the file and any think is wrong or not there let me knwo what is the right token for it

RESPONSE (key part)

The AI reviewed the token file against the sample program and pointed out the main missing or incorrect classifications. It said that quoted text such as `"expense"` and `"Rent"` should be `literal`, more specifically `string literal`. It also corrected `budget` to `keyword`, `rentLimit` and `spent` to `identifier`, `;` to `separator`, and `,` to `separator` instead of `operator`. The AI also noted that the table heading should ideally be `Lexeme | Token` rather than `Lexical | Token`.

ACCEPTED

I accepted the token corrections because they made the lexical classification more complete and more consistent with the sample program. The clarification about string literals was especially helpful.

REJECTED / MODIFIED

I kept my overall token breakdown, but I updated the missing token labels and corrected the misclassified ones based on the review. I also considered renaming the table heading from `Lexical` to `Lexeme` for better terminology.

ERRORS YOU CAUGHT

None.

REFLECTION

It is nice that AI can check my work and tell me where I did well and where I could do better. This helps me use AI as a reviewer while I am still learning how to classify tokens correctly.

Entry 7 of 8

Date	2026-05-07
Phase	**Parser**
AI tool	Codex (GPT-5)

Goal	Generate a BNF reference version of the grammar from the existing EBNF file for later comparison and use.

PROMPT

can u do the BNF for me i have the EBNF grammar in [D1_EBNF_grammar.txt](src/D1_EBNF_grammar.txt)

RESPONSE (key part)

The AI created a separate `D1_BNF_grammar.txt` file by converting the existing EBNF grammar into BNF for the same Part 1 subset. It explained that repetition was rewritten using helper rules such as tails, optional parts were replaced with nonterminals like `<optional_else>`, and empty choices were represented with `<empty>`.

ACCEPTED

I accepted the BNF file as a reference because it was generated directly from my EBNF file, so it should stay close to the same grammar design. It can help me later if I want to compare the two forms or understand how EBNF expansions work.

REJECTED / MODIFIED

I did not replace the EBNF file with the BNF file, because EBNF is still the main version needed for Part 1. I kept the BNF only as an extra reference.

ERRORS YOU CAUGHT

I did not review the BNF that the AI gave me, but I made it generate it from my EBNF file.

REFLECTION

I got the BNF without extra work, and this could help later. It is a good thing to have from AI.

Entry 8 of 8

Date	2026-05-07
Phase	**Writing**
AI tool	Codex (GPT-5)
Goal	Generate a full LaTeX draft for the Part 1 D1 PDF using the BudgetLang work already prepared.

PROMPT

okay generat the full laTex code to do the D1 pdf needed i will review the full pdf the pdf will take all the data needed from the files and the text i already made

RESPONSE (key part)

The AI created a full LaTeX file, `D1_P1_BudgetLang.tex`, containing the main Part 1 D1 sections for BudgetLang. The file included the language overview, lexical structure, syntax section with the EBNF grammar, names/binding/scope/lifetime discussion, and the sample program, all organized so the document could be compiled into a PDF.

ACCEPTED

I accepted the LaTeX draft because it collected the work I had already done into one report structure. This saved time and gave me a full draft that I can review and edit before submission.

REJECTED / MODIFIED

I did not treat the generated LaTeX as final without checking it. I planned to review the full PDF output and then fix or improve anything that needed changes.

ERRORS YOU CAUGHT

I will look at the full PDF generated by the AI and fix it if needed.

REFLECTION

AI can give you a full report from the work you already completed, and that helps a lot. After that, you can review all of it and modify anything if needed.

---

08 May 2026

Entry 1 of 4

Date	2026-05-08
Phase	**Testing**
AI tool	OpenAI Codex (GPT-5-based coding agent)
Goal	Create three different valid BudgetLang programs to use as parser test cases for Part 1 D3.

PROMPT

okay can u write me a 3 diffrent files to test them

RESPONSE (key part)

The AI created three valid BudgetLang test files in `src/tests/valid/`: `valid1.txt`, `valid2.txt`, and `valid3.txt`. It then ran all three through the parser with `--ast` and confirmed that each one parsed successfully and produced a printable AST.

ACCEPTED

I accepted the three generated test programs because they were meaningful BudgetLang programs and all of them parsed successfully when I ran them with the AST command. They gave me the three valid parser test cases I needed for D3.

REJECTED / MODIFIED

Nothing was rejected in this step. I kept the generated files as written because the parser accepted all three successfully.

ERRORS YOU CAUGHT

None.

REFLECTION

I learned that I should test every program the AI gives me. The AI helped me create valid test files quickly, but I still needed to run them myself and check the AST output.

Entry 2 of 4

Date	2026-05-08

Phase	**Design**
AI tool	Claude Opus 4.6
Goal	Ask the AI to generate an EBNF grammar for BudgetLang from a one-paragraph description (Experiment E1).

PROMPT

okay there is this :

E1 for Part 1:

Ask the AI to generate an EBNF grammar from a one-paragraph description of your language, then compare it with your own grammar and discuss what was wrong/suboptimal, especially precedence

required from me i already made my EBNF i want u to do the EBNF but do not look to my EBNF or BNF u are not allowed to look to the code or the PDF are in the Part_1_requirement other than that u can use

name the file AI_EBNF

RESPONSE (key part)

The AI read the lexical structure (`D1_lexical_structure.txt`), token list (`lexical&Token.txt`), and sample program (`small_draft_sample.txt`) — without looking at my EBNF, BNF, code, or requirement PDFs. It produced a complete EBNF grammar in `src/AI_EBNF.txt` with 6 sections: Program Structure, Type System, Statements, Expressions (6 precedence levels: comparison → additive → multiplicative → unary → member access → primary), Literals, and Lexical Elements. Key design decisions included making comparison non-associative, placing member access (`.`) above arithmetic in precedence, not supporting method calls on objects, and accepting general expressions for `budget`/`track` commands.

ACCEPTED

I accepted the full AI-generated EBNF file as-is because the purpose of E1 is to have an independent AI-generated grammar to compare against my own. The grammar covers all the constructs visible in the sample program and lexical structure.

REJECTED / MODIFIED

Nothing was rejected at this stage. The comparison and discussion of what is wrong or suboptimal will be done separately when I compare the AI grammar with my own.

ERRORS YOU CAUGHT

During this session, I did not verify the AI-generated grammar against my own EBNF yet, so I could not honestly claim specific errors at that point. I treated this grammar as a draft for Experiment E1, and I documented the concrete problems I found later in Entry 4 when I compared the two grammars directly.

REFLECTION

I learned that generating an independent AI grammar is useful only if I compare it carefully with my own design afterward. This session gave me a separate baseline for Experiment E1, but the real value came from the later comparison, not from accepting the generated grammar by itself.

Entry 3 of 4

Date	2026-05-08
Phase	**Writing**
AI tool	Claude Opus 4.6

Goal	Check whether the Part 1 D1 and D3 deliverables were ready for submission and identify any missing requirements or mistakes.

PROMPT

I want you to check @src/Part_1_requirement/210104004817_D1_P1.pdf and @src/Part_1_requirement/210104004817_D3_P1.pdf

if they are ready for submission. The project details and submission guid is under @"src/part 1". You may also need

to check the book for time to time the chapters are under @src/book_chapters. for book chapters and assignment

requirements please check md files, if you think there is something missning (md files are book chapters summary) you

may check the pdf chapters. Also you will review the code too under @src/BudgetLang

RESPONSE (key part)

The AI produced a submission-readiness review for Part 1. It identified some real issues, including a missing D5 retrospective, missing documentation for `//` comments in D1, an EBNF string-character rule that was too narrow compared with the lexer, and the need to mention the internal boolean result of comparison expressions. It also pointed out a static-scope risk in the type checker and interpreter for Part 2. However, it also made some incorrect or outdated claims, including saying that D4 did not have enough entries, that E1 was missing, and that the README test paths were wrong.

ACCEPTED

I accepted the parts of the review that identified real technical and documentation issues. These were useful because they helped me improve the D1 specification and notice a Part 2 static-scope risk in the implementation.

REJECTED / MODIFIED

I rejected the claims that D4 was under-populated, that E1 was missing, and that the README paths were wrong, because these did not match my actual files or the handout requirements. I checked the journal files, the README commands, and the submission rules before deciding what to keep from the review.

ERRORS YOU CAUGHT

Claude said E1 was missing, but I had already added an E1 entry in 08-05-2026.md.

REFLECTION

I learned that AI reviews can be useful, but I still need to verify them. This helped me compare the review with my real files.

Entry 4 of 4

Date	2026-05-08
Phase	**Design**
AI tool	Claude Opus 4.6 (Thinking)
Goal	Compare my hand-written EBNF grammar with the AI-generated EBNF grammar to evaluate which handles the language design better (Experiment E1 comparison).

PROMPT

there is a file name D1_EBNF_grammar.txt this is the one I wrote and this AI_EBNF.txt what the AI wrote. I want you to compare between them and I want to show who is doing it better and why. You will follow all of these files: D1_lexical_structure.txt, lexical&Token.txt, small_draft_sample.txt, book_chapters/MD-File-Version, book_chapters/PDF-File-Version, part 1.

RESPONSE (key part)

The AI produced a detailed 12-dimension comparison covering: program structure, type system (arrays), semicolon placement, expression precedence hierarchy (equality/relational split, unary minus, comparison associativity), assignment and lvalue design, expression statements, lexical definitions (string characters), documentation quality, LL(1) friendliness, and sample program coverage. It concluded the AI grammar won 7 dimensions to my 2, with 3 ties. Key AI grammar advantages cited were: unified `<lvalue>` rule, unary minus support, non-associative comparisons, factored `<type> ::= <base_type> [ "[" "]" ]`, and better documentation. My grammar's advantages were the finer equality/relational precedence split and the restrictive `<call_stmt>` that only allows function calls as standalone statements.

ACCEPTED

I accepted the structural observations — such as the missing unary minus in my grammar and the LL(1) disambiguation issue with my two separate assignment rules — because these are real technical gaps I can verify against Sebesta §3.3.1.8 and §4.4.2.

REJECTED / MODIFIED

I rejected the overall conclusion that the AI grammar was clearly "better." Some of the AI grammar's so-called improvements — like making all types arrayable with `<base_type> [ "[" "]" ]` — are more general, but not necessarily better for my Part 1 subset, where only `transaction[]` is used. The AI described my grammar unfairly by treating generality as automatically superior without considering that my narrower rules are deliberate design choices for a domain-specific language. Accepting the AI's verdict without checking would have undermined my own design rationale.

ERRORS YOU CAUGHT

The AI said the AI grammar was better, but when I checked the grammar files i saw that some of its improvements make it more general and not actually better for my Part 1.

If I had accepted what the AI said without checking, it would have described my own grammar unfairly.

REFLECTION

I learned that AI comparisons can sound convincing even when they overstate their conclusions.

This session helped me see that I need to compare the AI claims with my actual grammar rather than taking the verdict at face value.
